PRACTICA I

Mecatrónica
ITLA
La Caleta,
Santo Domingo
20240712@itla.edu.do
Cristian Miguel Jimenez

***Resumen*— En este practica, aprendimos muchas cosas, entre ellas a como crear compuertas and y or con compuertas nand o nors, ademas mejoramos nuestra habilidades con el lenguaje de verilog.**

***Palabras claves*— compuertas, verilog.**

I. Introducción

El objetivo de esta práctica es fortalecer la destreza en el uso de compuertas lógicas y el lenguaje Verilog. A través de la resolución de ejercicios, demostraremos la transición de un diagrama lógico a su respectiva tabla de verdad, comprobando que cualquier planteamiento lógico puede ser materializado mediante hardware.

II. Marco Teorico

A. *Compuertas logicas*

Las compuertas lógicas constituyen las unidades básicas de la electrónica digital. Estos dispositivos procesan señales de entrada mediante operaciones lógicas para generar una respuesta específica, permitiendo que las funciones booleanas se materialicen en componentes físicos de hardware dentro de sistemas computacionales.

B. *Verilog*

Verilog se define como un lenguaje de descripción de hardware (HDL) orientado al modelado y validación de sistemas digitales. Su función principal es facilitar el diseño de arquitecturas complejas, como FPGAs y ASICs, permitiendo especificar tanto la interconexión estructural como el comportamiento lógico de los componentes electrónicos.

III. Montaje Experimental

A. *Practica I*

Se utilizaron los siguientes materiales para realizar el experimento:

- FPGA TANG PRIMER 25K
- Cable Usb Tipo A a C

Estos fueron las herramientas o materiales que utilice para realizar los ejercicios practicos en fisico que veremos mas adelante en el documento.

Bueno, demos inicio al desarrollo de está practica, empecemos con el mandato del primer ejercicio:

1. El objetivo es el diseño de un sistema combinacional con un bus de entrada de 4 bits (X_3, X_2, X_1, X_0) y una salida única (S). La función de salida debe presentar un nivel lógico alto (1) bajo tres condiciones específicas: cuando todas las líneas de entrada se encuentran en alto, cuando todas están en bajo (estado nulo), o cuando la cantidad de entradas activas sea un número impar.

a) Obtenga la tabla de verdad.

A	B	C	D	S1
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Fig. 1 Tabla de Verdad

Las salidas de esta tabla se obtuvieron identificando las combinaciones impares en la entrada de 4 bits, además de considerar los casos ya mencionados en los que todos los bits se encuentran en 0000 o 1111..

b) Encuentre la función de salida simplificada en suma de producto (SoP).

Minterminos

S1 = (A´ B´ C´ D´) + (A´ B´ C´ D) + (A´ B´ C D´) + (A´ B C´ D´) + (A´ B C D) + (A B´ C´ D´) + (A B´ C D) + (A B C´ D) + (A B C D´) + (A B C D)

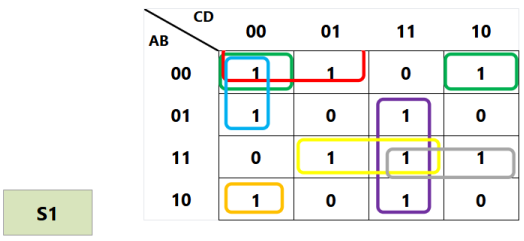


Fig. 2 K-maps minterminos S1

S1

S1 = (A´ B´ C´)
S1 = (A´ B´ D´)
S1 = (A´ C´ D´)
S1 = (B´ C´ D´)
S1 = (B C D)
S1 = (A C D)
S1 = (A B D)
S1 = (A B C)

c) Encuentre la función de salida simplificada en producto de suma (PoS).

Maxterminos

S1 = (A´+ B´+ C + D) (A´+ B + C´+ D) (A´+ B + C + D´) (A + B ´+ C´+ D) (A + B´+ C + D´) (A + B + C´+ D´)

S1

AB \ CD	00	01	11	10
00	1	1	0	1
01	1	0	1	0
11	0	1	1	1
10	1	0	1	0

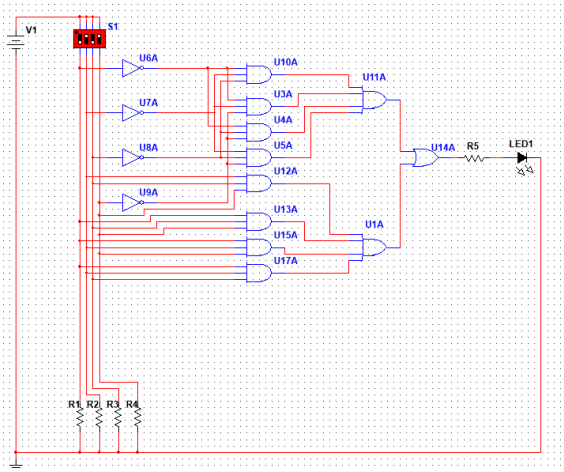
Fig. 3 K-maps maxterminos S1

S1

S1 = (A´ B´ C D)
S1 = (A´ B C´ D)
S1 = (A´ B C D´)
S1 = (A B´ C´ D)
S1 = (A B´ C D´)
S1 = (A B C´ D´)

d) Use software de simulación para simular diagrama lógico.

Al final hice el daigrama logico para Min y Max terminos pero me decidí por colocar el de Min terminos ya que quiero trabajar mas con este para enterdelo mejor.



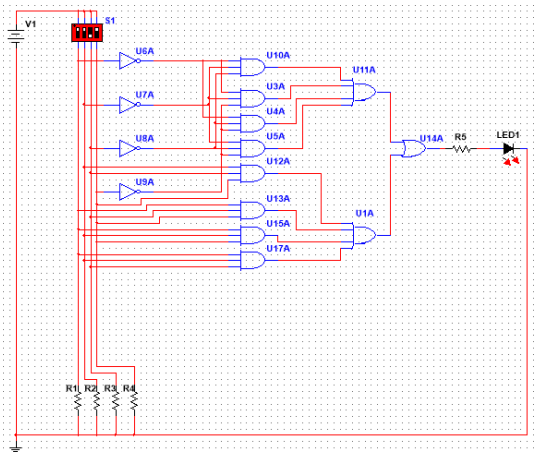


Fig. 5 Diagrama logico Minterminos en Multisim

2. Se desea diseñar un circuito de 5 bits que siga la sucesión de Fibonacci en su salida.

a) Obtenga la tabla de verdad.

A	B	C	D	E	S1
0	0	0	0	0	1
0	0	0	0	1	1
0	0	0	1	0	1
0	0	0	1	1	1
0	0	1	0	0	0
0	0	1	0	1	1
0	0	1	1	0	0
0	0	1	1	1	0
0	1	0	0	0	1
0	1	0	0	1	0
0	1	0	1	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	0	1	1
0	1	1	1	0	0
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	0	1	0
1	0	0	1	0	0
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	0	1	1
1	0	1	1	0	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	0	1	0
1	1	0	1	0	0
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	0

Fig. 6 Tabla de Verdad

b) Encuentre la función de salida simplificada en suma de producto (SoP).

Minterminos

S1 = (A´ B´ C´ D´ E´) + (A´ B´ C´ D´ E) + (A´ B´ C´ D E´) + (A´ B´ C´ D E) + (A´ B´ C D´ E) + (A´ B C´ D´ E´) + (A´ B C D´ E) + (A B´ C D´ E)

S1

ABC \ CD	00	01	11	10
000	1	1	1	1
001	0	1	0	0
011	0	1	0	0
010	1	0	0	0
100	0	0	0	0
101	0	1	0	0
111	0	0	0	0
110	0	0	0	0

Fig. 7 K-maps Minterminos

S1

S1 = (A´ B´ C´)

S1 = (A´ C´ D´ E´)

S1 = (A´ C D´ E´)

S1 = (B´ C D´ E)

c) Use software de simulación para simular diagrama lógico.

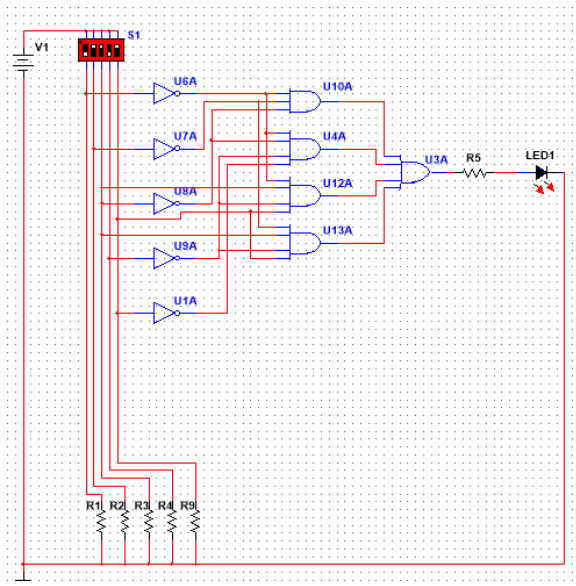


Fig. 8 Diagrama logico Minterminos en Multisim

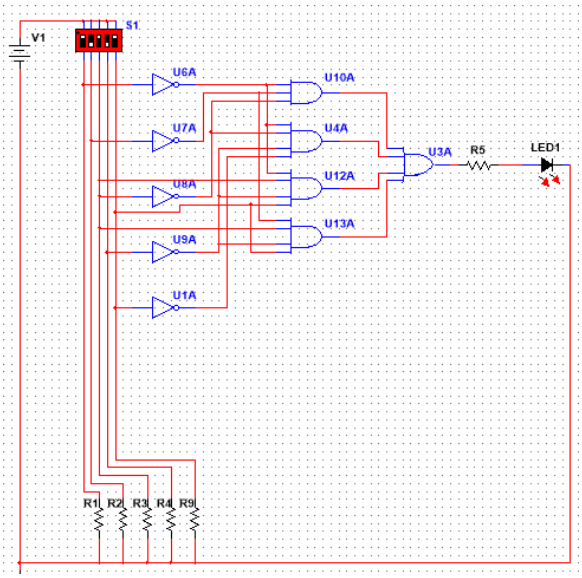
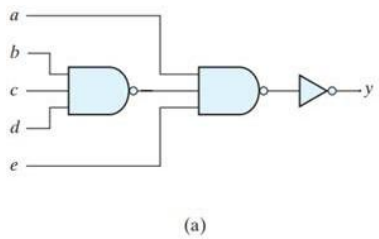


Fig. 9 Diagrama logico Minterminos en Multisim

3. De los siguientes diagramas lógicos deduzca la tabla de verdad y escriba la función booleana resultante.



(a)

Fig. 10 Ejercicio 3, parte A

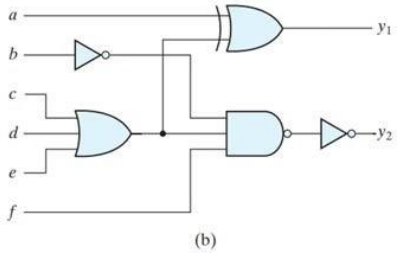
A	B	C	D	E	Y
0	0	0	0	0	0
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	0
0	0	1	0	0	0
0	0	1	0	1	0
0	0	1	1	0	0
0	0	1	1	1	0
0	1	0	0	0	0
0	1	0	0	1	0
0	1	0	1	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	0	1	0
0	1	1	1	0	0
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	0	1	1
1	0	0	1	0	0
1	0	0	1	1	1
1	0	1	0	0	0
1	0	1	0	1	1
1	0	1	1	0	0
1	0	1	1	1	1
1	1	0	0	0	0
1	1	0	0	1	1
1	1	0	1	0	0
1	1	0	1	1	1
1	1	1	0	0	0
1	1	1	0	1	1
1	1	1	1	0	0
1	1	1	1	1	0

Fig. 11 Tabla de Verdad

Ahora pasaremos a poner la funcion booleana resultante:

Minterminos

Y1 = (A B´ C´ D´ E) + (A B´ C´ D E) + (A B´ C D´ E) + (A B C´ D ´ E) + (A B C´ D E) + (A B C D´ E)



(b)

Fig. 12 Ejercicio 3, parte B

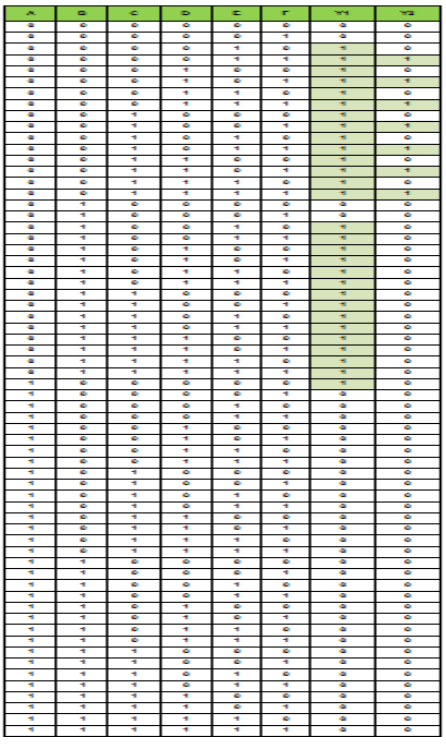


Fig. 13 Tabla de Verdad

Ahora pasaremos a poner la funcion booleana resultante:

Minterminos

Y1 = (A´ B´ C´ D´ E F´) + (A´ B´ C´ D´ E F) + (A´ B´ C´ D E´ F´) + (A´ B´ C´ D E´ F) + (A´ B´ C´ D E F´) + (A´ B´ C D´ E´ F´) + (A´ B´ C D´ E´ F) + (A´ B´ C D´ E F) + (A´ B´ C D E´ F´) + (A´ B´ C D E F) + (A´ B C´ D´ E F´) + (A´ B C´ D´ E F) + (A´ B C´ D´ E F´) + (A´ B C´ D´ E´ F´) + (A´ B C´ D E´ F´) + (A´ B C´ D E F´) + (A´ B C´ D E F) + (A´ B C D´ E´ F´) + (A´ B C D´ E´ F) + (A´ B C D´ E F´) + (A´ B C D´ E F) + (A´ B C D E´ F´) + (A´ B C D E´ F) + (A´ B C D E F´) + (A´ B C D E F) + (A B´ C´ D´ E´ F´) + (A B´ C´ D´ E´ F)

Y2 = (A´ B´ C´ D´ E F) + (A´ B´ C´ D E´ F) + (A´ B´ C´ D E F) + (A´ B´ C D´ E´ F) + (A´ B´ C D´ E F) + (A´ B´ C D E´ F) + (A´ B´ C D E F)

4. Del siguiente diagrama de tiempo deduzca la tabla de verdad y la función booleana complementada, siendo A, B y C los bits de entrada y Fs la función de salida. (tome los picos de las entradas y la salida como interferencias indeseadas).

A	B	C	FS
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Fig. 14 Tabla de Verdad

Ahora pasaremos a poner la funcion booleana:

Minterminos

FS = (A´ B´ C´) + (A´ B C) + (A B´ C) + (A´ B´ C)

FS COMPLEMENTADA = (A + B + C) (A + B´+ C´) (A´+ B + C´) (A + B + C´)

5. Es necesario multiplicar dos números binarios, cada uno de dos bits para formar su producto en binario. Asuma los dos números representados por A1, A0 y B1 y B0 donde el suscrito 0 denote el bit menos significativo.

a) Determine el número de líneas salidas necesarios.

Ya que son números binarios, cada uno de dos bits, las salidas serian 4 bits.

b) Encuentre las expresiones de Boole simplificadas para cada salida.

Ya que son números binarios, cada uno de dos bits, las salidas serian 4 bits.

S1 = (A1) (B1)

S2 = (A1 B1) ⊕ (A0 B0)

S3 = (A1´ A0 B1) + (A0 B1 B0´) + (A1 A0´ B0) + (A1 B1´ B0) S4 = (A0 B0)

6. De la siguiente expresión lógica booleana simular:

$$Z = \overline{((A + B).C).D}$$

Fig. 15 Funcion

a) Tal como se presenta la función.

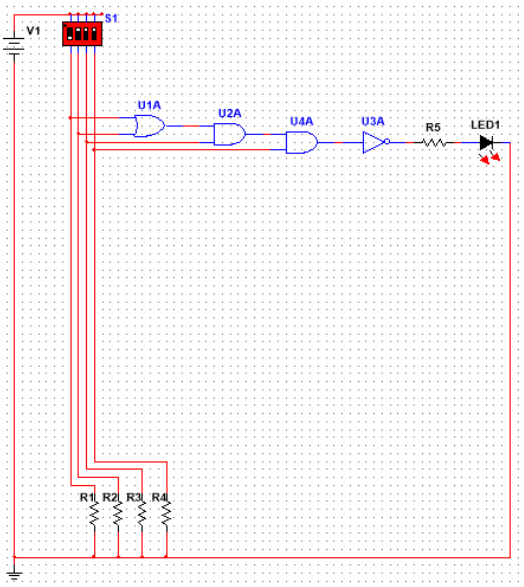


Fig. 16 Diagrama logico en Multisim

b) Usando compuertas dual line Nands.

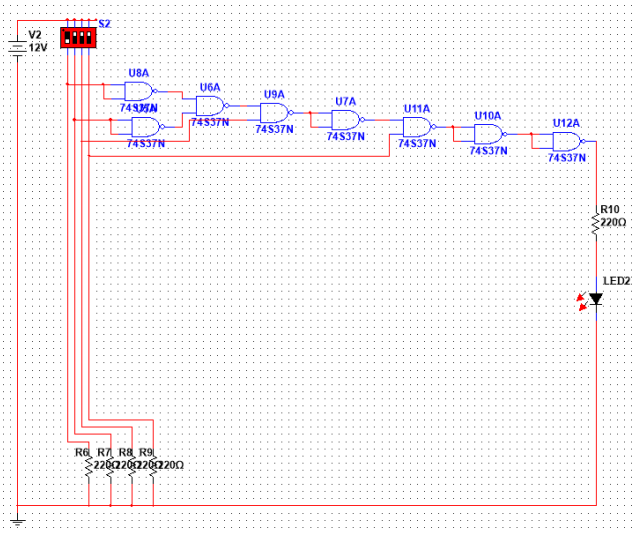


Fig. 17 Diagrama logico en Multisim

c) Usando compuertas dual line Nors.

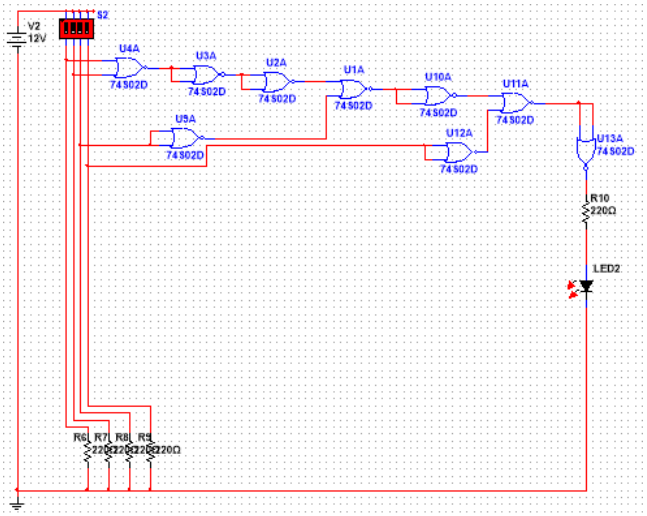


Fig. 18 Diagrama logico en Multisim

7. Realice un circuito combinacional de cuatro bits que complemente a dos su entrada y una salida extra para dejar saber si el complemento es positivo (Nivel alto) o negativo (Nivel bajo) Tome el complemento del número cero como cero.

a) Obtenga la tabla de verdad.

A	B	C	D	SIGNO	Ca	Cb	Cc	Cd
0	0	0	0	0	0	0	0	0
0	0	0	1	1	1	1	1	1
0	0	1	0	1	1	1	1	0
0	0	1	1	1	1	1	0	1
0	1	0	0	1	1	1	0	0
0	1	0	1	1	1	0	1	1
0	1	1	0	1	1	0	1	0
0	1	1	1	1	1	0	0	1
1	0	0	0	1	1	0	0	0
1	0	0	1	1	0	1	1	1
1	0	1	0	1	0	1	1	0
1	0	1	1	1	0	1	0	1
1	1	0	0	1	0	1	0	0
1	1	0	1	1	0	0	1	1
1	1	1	0	1	0	0	1	0
1	1	1	1	1	0	0	0	1

Fig. 19 Tabla de Verdad

b) Encuentre la función de salida simplificada en producto de suma (PoS)

SIGNO

AB \ CD	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

Fig. 20 K-maps SIGNO

Ca

AB \ CD	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	0	0	0	0
10	1	0	0	0

Fig. 21 K-maps Ca

Cb

AB \ CD	00	01	11	10
00	0	1	1	1
01	1	0	0	0
11	1	0	0	0
10	0	1	1	1

Fig. 22 K-maps Cb

Cc

AB \ CD	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	0	1	0	1
10	0	1	0	1

Fig. 23 K-maps Cc

Cc

AB \ CD	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	0	1	0	1
10	0	1	0	1

Fig. 24 K-maps Cd

Ahora pasaremos a poner la funcion booleana:

SIGNO

SIGNO = (A´+ B´+ C´+D´)
SIGNO = (A´+ B´+ C´+D´)

Ca

Ca = (A´+ B´+ C´+ D´)
Ca = (A + C´+ D)
Ca = (A + B´+ C)
Ca = (A + B)
Ca = (A´+ B´+ C´+ D´) (A + C´+ D) (A + B´+ C) (A + B)

Cb

Cb = (B´+ C´+ D´)
Cb = (B + D)
Cb = (B + C)
Cb = (B´+ C´+ D´) (B + D) (B + C)

Cc

Cc = (C´+ D´)
Cc = (C + D)
Cc = (C´+ D´) (C + D)

Cd

Cd = (C´+ D´)
Cd = (A´+ B´+ C´)
Cd = (B + C + D´)
Cd = (A + C + D´)
Cd = (C´+ D´) (A´+ B´+ C´) (B + C + D´) (A + C + D´)

8. Implementar el siguiente circuito usando HDL.

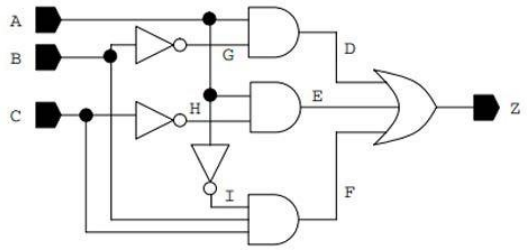


Fig. 25 Diagrama logico

a) Código de verilog.

```
module ejercicio7(input A, B, C, D, output SIGNO, output Ca, Cb, Cc, Cd);

wire [12:0] o;
wire [5:0] n;
wire [4:0] a;

// Negaciones de entradas
not (n[0], A);
not (n[1], B);
not (n[2], C);
not (n[3], D);

// Compuertas para la salida SIGNO
or (a[0], n[0], n[1], n[2], n[3]);
not (SIGNO, a[0]);

// Compuertas para la salida Ca
or (o[0], n[0], n[1], n[2], n[3]);
or (o[1], A, n[2], D);
or (o[2], A, n[1], C);
or (o[3], A, B);
and (a[1], o[0], o[1], o[2], o[3]);
not (Ca, a[1]);
```

```
// Compuertas para la salida Cb
or (o[4], n[1], n[2], n[3]);
or (o[5], B, D);
or (o[6], B, C);
and (a[2], o[4], o[5], o[6]);
not (Cb, a[2]);

// Compuertas para la salida Cc
or (o[7], n[2], n[3]);
or (o[8], C, D);
and (a[3], o[7], o[8]);
not (Cc, a[3]);

// Compuertas para la salida Cd
or (o[9], n[2], n[3]);
or (o[10], n[0], n[1], n[2]);
or (o[11], B, C, D);
or (o[12], A, C, n[3]);
and (a[4], o[9], o[10], o[11], o[12]);
not (Cd, a[4]);

endmodule
```

b) Test bench.

```
`timescale 1ns / 1ps  // Definimos el tiempo de simulación: 1ns de unidad de tiempo y 1ps de precisión

module testbench;

    // Definimos las señales de entrada (registros, ya que son variables de simulación)
  reg A, B, C, D;

   // Definimos las señales de salida (wires, ya que son salidas del módulo)
  wire SIGNO, Ca, Cb, Cc, Cd;

  // Instanciamos el módulo ejercicio7
  ejercicio7 uut (
    .A(A),
    .B(B),
    .C(C),
    .D(D),
    .SIGNO(SIGNO),
    .Ca(Ca),
    .Cb(Cb),
    .Cc(Cc),
    .Cd(Cd)
  );

  // Bloque inicial para aplicar las combinaciones de entradas
  initial begin
    // Inicializamos las señales de entrada
    A = 0; B = 0; C = 0; D = 0;
    #10;  // Esperamos 10 unidades de tiempo

// Aplicamos todas las combinaciones posibles de A, B, C, D
      A = 0; B = 0; C = 0; D = 1; #10;
      A = 0; B = 0; C = 1; D = 0; #10;
      A = 0; B = 0; C = 1; D = 1; #10;
      A = 0; B = 1; C = 0; D = 0; #10;
      A = 0; B = 1; C = 0; D = 1; #10;
      A = 0; B = 1; C = 1; D = 0; #10;
      A = 0; B = 1; C = 1; D = 1; #10;
      A = 1; B = 0; C = 0; D = 0; #10;
      A = 1; B = 0; C = 0; D = 1; #10;
      A = 1; B = 0; C = 1; D = 0; #10;
      A = 1; B = 0; C = 1; D = 1; #10;
      A = 1; B = 1; C = 0; D = 0; #10;
      A = 1; B = 1; C = 0; D = 1; #10;
      A = 1; B = 1; C = 1; D = 0; #10;
      A = 1; B = 1; C = 1; D = 1; #10;

    // Fin de la simulación
    $finish;
  end

  // Monitorizamos las señales de salida para observar el comportamiento
  initial begin
    $monitor("A=%b B=%b C=%b D=%b | SIGNO=%b Ca=%b Cb=%b Cc=%b Cd=%b",
             A, B, C, D, SIGNO, Ca, Cb, Cc, Cd);
  end

  // Inicialización de las señales y visualización de la simulación
  initial begin
    $display("Simulación iniciada. Observando las salidas...");
  end

endmodule
```

c) Simulación en software (diagrama lógico).

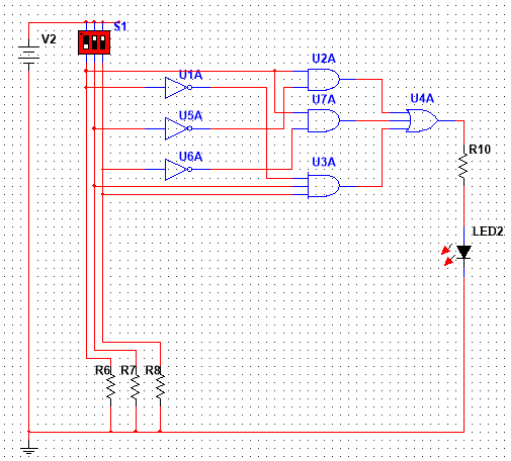


Fig. 26 Diagrama logico en Multisim

9. Diseñe un detector de paridad para un número de 7 bits, incluido el bit de paridad.

a) Diagramas lógico en multisim.

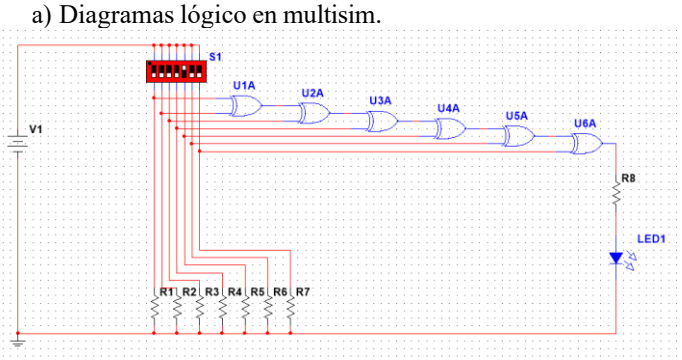


Fig. 27 Diagrama logico en Multisim

b) Código de verilog.

```
module ejer8 (input A, B, C, // Entradas
output Z // Salida
);

wire notB, notA, NotC, AB, AC, ABC;

not(notA,A);
not(notB,B);
not(NotC,C);
and(AB,A,notB );
and(AC,A,NotC);
and(ABC,notA,B,C);
or(Z,AB,ABC,AC);

endmodule
```

10. El acceso a la rotonda está controlado por un semáforo ubicado en cada una de las cuatro vías (ver la figura), además de un sensor que determina si la vía respectiva tiene vehículos esperando para entrar a la rotonda. En la figura, SO, S1, S2 y S3 indican los semáforos y P0, P1, P2, P3 indican los sensores de las vías. El estado de los semáforos está determinado por las siguientes condiciones: Cuando uno de los sensores se activa (es decir hay vehículos que desean entrar a la glorieta) entonces el semáforo de esta vía debe estar en luz verde (es decir permite el paso de vehículos) y el semáforo anterior debe estar en luz roja es decir no se permite el paso de los vehículos), los demás pueden estar en luz verde. Un ejemplo de esta situación es la siguiente: si P1 esta activado entonces S1 se pone en verde, S0 debe ponerse en rojo y los demás pueden ponerse en verde. Si 2 o más sensores se activan al mismo tiempo, debe darse prioridad a la vía cuyo semáforo Sn está indicado con el menor número. Un ejemplo de esta situación es la siguiente: si P2 y P1 se activan al mismo tiempo, entonces S2 debe ponerse en rojo y S1 en verde, los demás pueden estar en verde. De acuerdo con lo anterior, la vía con mayor prioridad es la que posee el semáforo S0. Puede indicarse un semáforo con una sola luz de tal forma que si la luz está encendida entonces el semáforo está en verde. Si la luz está apagada entonces el semáforo está en rojo. Las entradas al sistema digital es el estado de los sensores, mientras que las salidas son el estado de los semáforos.

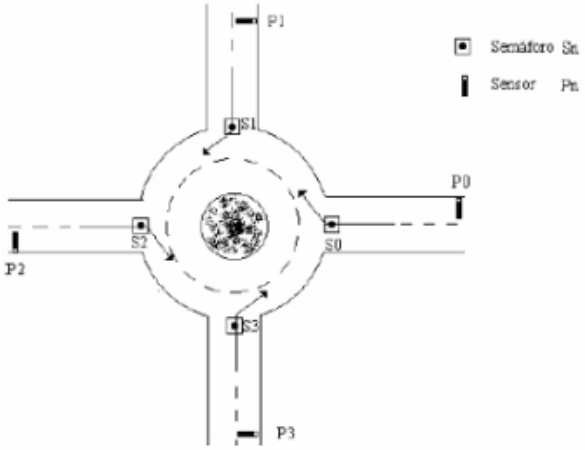


Fig. 28 Ejercicio 10

a) Obtenga la tabla de verdad.

P0	P1	P2	P3	S0	S1	S2	S3
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	1
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	1
1	0	1	0	1	0	1	0
1	0	1	1	1	0	1	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	0	1	0	1	0
1	1	1	1	1	0	1	0

Fig. 29 Tabla de Verdad

b) Obtenga la tabla de verdad.

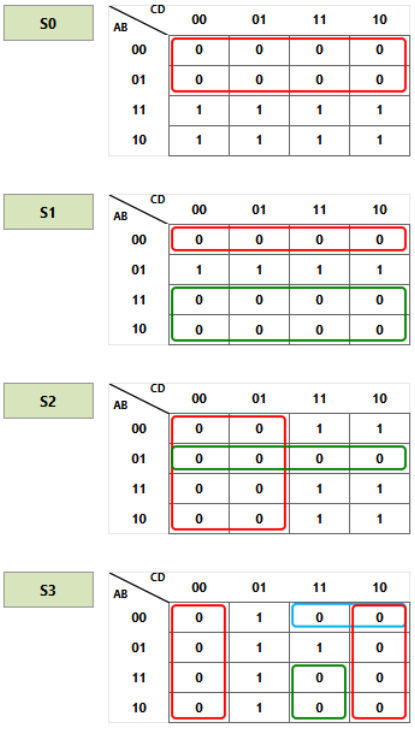


Fig. 30 K-maps

S0

S0 = (A´)
S0 = (A´)

S1

S1 = (A´+ B´)
S1 = (A)
S1 = (A´+ B´) (A)

S2

S2 = (C´)
S2 = (A´+ B)
S2 = (C´) (A´+ B)

S3

S3 = (D´)
S3 = (A + C)
S3 = (A´+ B´+ C)
S3 = (D´) (A + C) (A´+ B´+ C)

c) Diagramas lógico simulado .

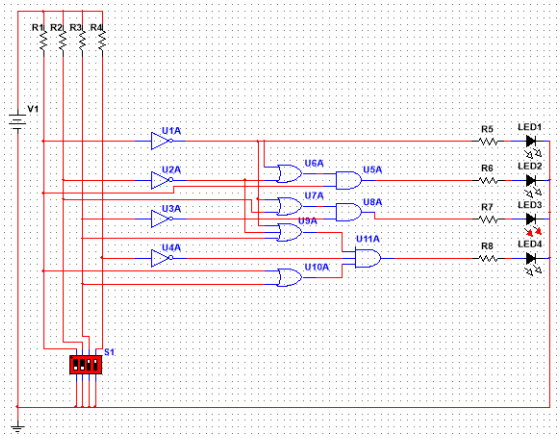


Fig. 31 Diagrama logico en Multisim

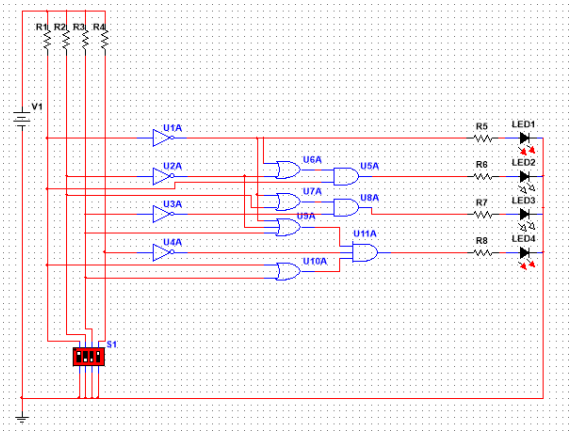


Fig. 32 Diagrama logico en Multisim

d) Código HDL.

```
module Semaforo(input P0, P1, P2, P3, output S0, S1, S2, S3);

wire [4:0]o;
wire [3:0]n;
wire [3:0]a;

//Negaciones de las entradas
not (n[0], P0);
not (n[1], P1);
not (n[2], P2);
not (n[3], P3);

//Compuertas para S0
or (a[3], n[0]);
not(S0, a[3]);

//Compuertas para S1
or (o[0], n[0], n[1]);
and (a[0], o[0], P0);
not (S1, a[0]);

//Compuertas para S2
or (o[1], n[0], P1);
and (a[1], o[1], n[2]);
not (S2, a[1]);

//Compuertas para S3
or (o[2], P0, P2);
or (o[3], n[0], n[1], P2);
and (a[2], o[2], o[3], n[3]);
not (S3, a[2]);

endmodule.
```

IV. Conclusión

En esta práctica, hemos profundizado en el diseño y análisis de circuitos combinacionales utilizando compuertas lógicas y el lenguaje de descripción de hardware Verilog. A lo largo de los ejercicios, aprendimos a transformar un diagrama lógico en una tabla de verdad, lo cual es fundamental para entender el comportamiento de un circuito antes de implementarlo en hardware.

A medida que avanzamos, pudimos observar que cualquier operación o funcionalidad lógica que deseemos implementar puede ser expresada mediante una combinación adecuada de compuertas lógicas. Este concepto es clave, ya que nos permite modelar de manera eficiente y precisa cualquier tipo de sistema digital.

Además, el uso de Verilog como herramienta para describir y simular estos circuitos nos permitió afianzar nuestros conocimientos sobre la programación y simulación de circuitos digitales. Gracias a Verilog, podemos representar de manera compacta y modular cualquier diseño, facilitando la verificación y validación de los mismos mediante simulaciones.

Video
https://drive.google.com/drive/folders/1WLb3rgP94iZKghVTCB2ftSsvkZduk4ij?usp=drive_link